{EPITECH}

BOOTSTRAP_

< OLLAMA PYTHON SCRIPT />



BOOTSTRAP

Introduction

Sensai is an open-scope project, which means the freedom is real, and so is the risk that comes with it. Your main hurdle will be building before you finish thinking, accumulating features that do not connect, and putting together a demo that works in parts but does not tell a coherent story. The bootstrap is designed to help you avoid that.

This bootstrap is separated in two parts, feel free to do them (or not) in any order.

Ollama chatbot MVP gets your environment running. Ollama needs to be installed, a model needs to be pulled, and every member of the group needs to be able to run the mandatory CLI chatbot on their own machine before the real work begins.
Scoping your project gives you tools to make good collective decisions about what you are building. How to run a useful brainstorm without converging on the first idea that sounds reasonable. How to define a use case specific enough to actually constrain your choices. How to map your features before you commit to them. How to stress-test your scope before you start coding.

Ollama chatbot MVP

In this section, your goal is to build the mandatory part of the Sensai project.

At the end of this exercise, your chatbot should satisfy the following requirements:

- ✓ Launched from the command line with a model name as argument or config
- ✓ Responses stream progressively (character by character or chunk by chunk), not as a single block
- ✓ Conversation history is maintained within the session (prior turns are sent as context to the model)
- ✓ A configurable system prompt is loaded from a file, not hardcoded
- ✓ Errors are handled explicitly: unavailable model, empty input, connection failure

Feel free to follow those steps, or any other online ressource.

Step 1 — Install Ollama

Ollama runs language models locally on your machine. It exposes a simple HTTP API that your Python code will call directly.

```
curl -fsSL https://ollama.com/install.sh | sh
```

Step 2 — Pull a model

Two options to select and pull a model.

1. From Ollama list of models

```
ollama pull MODEL_NAME
pulling manifest...
pulling model... 100%
```

2. From hugging face (via Ollama), see Tutorial.
   Then filter and select from HF model list



Some model weights are several gigabytes. Be mindful of your machine and disk space when selecting a model. Prefer a lightweight model for the initial MVP to ensure fast iteration and stable performance. Larger models can be tested later once the system is fully functional.

A smaller model typically has fewer parameters, which often improves speed and responsiveness on local machines, at the cost of reasoning capacity and output quality depending on the task.

Step 3 — Verify Ollama is running

Ollama runs as a background service. Your Python code communicates with it at `http://localhost:11434`.

Check it is up:

```
Terminal
$> curl http://localhost:11434
Ollama is running
```

If you get a connection error, start the service manually:

```
Terminal
$> ollama serve
```

Step 4 — Set up your Python environment

Python 3.10 or higher is required. Check your version, and then **create a virtual environment:**

```
Terminal
$> python3 -m venv .venv
source .venv/bin/activate
```

You only need the `requests` library to get started. Add others as your feature set grows.



Create a `requirements.txt` now and keep it up to date throughout the project:



Step 5 — Make your first API call

```python
import requests
import json

response = requests.post(
    "http://localhost:11434/api/chat",
    json={
        "model": "llama3",
        "messages": [
            {"role": "user", "content": "Reply with exactly three words."}
        ],
        "stream": False
    }
)

data = response.json()
print(data["message"]["content"])
```

The `messages` list is your context. Every time you append a turn to it and send the full list to the model, the model can see the conversation history.



Step 6 — Build the rest of the owl

Go on now. You can do it.

Step 7 — Verify your MVP

Before moving on to optional features, confirm your chatbot satisfies all mandatory requirements. Go back to the beginning of the subject

{”role”: ”user”, ”content”: ”What would be a good list of features to check for for the exercise I'm doing?”}

{EPITECH}

Scoping your project

Your MVP is the foundation. Every feature in the catalogue plugs into this loop. But before choosing your features, read the full subject and the feature catalogue carefully. The features you choose should form a coherent system, not a list of independent additions. That decision deserves as much rigor as the code itself.

Tool 1 — Individual exploration before collective decision

One of the most reliable ways to waste time in a group is to discuss options before everyone has thought about them independently. The loudest voice wins, and the group converges on the first idea that sounds reasonable rather than the best one.

A better approach:

Before your first scoping meeting, each group member independently reads the feature catalogue and answers questions in writing without consulting the others:

- ✓ What is the most interesting technical problem in this catalogue for me personally?
- ✓ If this system were a real product, what would it be for? Who uses it, and in what context?
- ✓ Which three features would I combine to build something coherent around that use case?
- ✓ Which feature am I most afraid of technically? Why?



This question list is indicative. A good project manager will select the ones most relevant for the context, and add some of their own.

Then meet. Compare answers.
Look for overlaps: they are your strongest candidates.
Look for disagreements: they are worth debating.

Tool 2 — Define your use case before your feature list

The Sensai project requires that you write a list of user stories integrated in a real-world context. This "context" is a use case for your application, and should be decided on early in the project. It should answer three questions:

- ✓ Who is using the system?
- ✓ What are they trying to accomplish?
- ✓ Why can't they do it without your system?

A good use case is specific enough to constrain your feature choices. If any feature in the catalogue could plausibly fit your use case, your use case is too vague.

> **Strong use case:** "A developer working alone on a long-term project who needs an assistant that remembers the codebase context across sessions, can execute and debug code autonomously, and explains its reasoning transparently."

This immediately suggests features: persistent memory, sandboxed execution, visible reasoning, structured logging. It also immediately rules out features that don't fit: multi-user sharing, cron scheduling, web UI.



Is this a decision you can take alone, or should there be a brainstorm for it too?

Tool 3 — Stress-test your scope

Before locking in your feature set, apply these tests:

The demo test. Imagine your keynote. Can you tell a coherent story with these features? Is there a scenario where a real person uses your system and each feature plays a visible role? If you struggle to construct that story, your feature set might lack coherence.

The time test. For each feature, estimate honestly: how long will it take to implement it well (including edge cases, testing, liking user stories...)? Sum those estimates. Add some percentage of time for integration depending on how many interfaces you will need to align.. Does that fit your timeline?

The depth test. Pick your hardest feature. Can every member of the group explain, at a technical level, how it works? If the answer is no, you either need to study it before committing or replace it with something your group can own fully.

The cut test. If you had to remove one feature the week before the demo, which one would hurt the least? That feature might not need to be in your scope in the first place.

Tool 4 — Write your architecture before your code

Once your feature set is locked, sketch your architecture before opening your editor. Not a full technical specification — a rough map of the components you will build and how they connect.



You whole Tek2 has prepared you for this moment.

This exercise has two benefits. First, it surfaces integration problems early, when they are cheap to fix. Second, it gives your group a shared mental model of the system, so you can divide work without creating components that cannot be assembled.

A component with unclear responsibility or too many dependencies is a warning sign. Redesign it before building it.

Before you start building

When your group has completed this section, you should have:

- ✓ A **written use case:** specific enough to constrain your feature choices
- ✓ A **feature list:** justified against the use case, mapped for dependencies
- ✓ A **rough architecture:** components, responsibilities, interfaces

None of this needs to be formal. A shared document, a whiteboard photo, a short README section. What matters is that every member of the group is aligned with them and can project themselves in the final vision of the project.

> Who is your system for?
> What features are you building, and why those?
> How do the components connect?

If you can answer those questions clearly, you are ready to build.

v 0.1

